

LAPORAN PRAKTIKUM PEKAN 6

STRUKTUR DATA

Disusun Oleh:

Muhammad Fharel

2511531010

Dosen Pengampu:

Dr. Wahyudi S.T.M.T

Asisten Pratikum:

Muhammad Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Segala puji penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga laporan praktikum Struktur Data pada tanggal 12 Mei 2026 yang membahas tentang bagaimana struktur data *Double Linked List* bekerja serta bagaimana mengimplementasikan berbagai operasi seperti penambahan, penelusuran, dan penghapusan *node*. Penulis juga memahami peran penting *pointer* ganda (*next* dan *prev*) dalam menghubungkan setiap *node*.

Ucapan terima kasih ditujukan kepada dosen pengampu, asisten praktikum, serta rekan-rekan yang telah membantu dalam proses pelaksanaan praktikum. Penulis menyadari bahwa penulisan laporan masih jauh dari kata sempurna, oleh karena itu kritik dan saran sangat diharapkan untuk penyempurnaan di kemudian hari. Semoga laporan ini memberikan manfaat dan menambah wawasan pembaca.

Padang, 14 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	3
2.1 Double Linked List.....	3
2.2 Kode Pemograman	3
2.2.1 Class NodeDLL.....	3
2.2.2 Class InsertDLL	4
2.2.3 Output InsertDLL.....	6
2.2.4 Class PenelusuranDLL.....	7
2.2.5 Output PenelusuranDLL	8
2.2.6 Class HapusDLL	8
2.2.7 Output HapusDLL.....	11
BAB III PENUTUP	12
3.1 Kesimpulan.....	12
DAFTAR PUSTAKA	13
TUGAS PRAKTIKUM.....	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, kebutuhan untuk mengelola data secara dinamis semakin meningkat, terutama ketika data sering mengalami perubahan seperti penambahan dan penghapusan. Salah satu struktur data yang dapat digunakan untuk mengatasi hal tersebut adalah *Double Linked List*.

Double Linked List merupakan pengembangan dari *Single Linked List*, di mana setiap *node* memiliki dua *pointer*, yaitu *pointer* ke *node* berikutnya (*next*) dan *pointer* ke *node* sebelumnya (*prev*). Hal ini memungkinkan proses penelusuran dilakukan secara dua arah, sehingga lebih fleksibel dibandingkan *Single Linked List*.

Pada praktikum ini, dilakukan implementasi berbagai operasi pada *Double Linked List* seperti penambahan *node* di awal, akhir, dan posisi tertentu, penelusuran maju dan mundur, serta penghapusan *node*. Dengan demikian, mahasiswa dapat memahami cara kerja struktur data ini secara lebih mendalam.

1.2 Tujuan Praktikum

1. Memahami konsep dasar *Double Linked List*.
2. Mengetahui struktur *node* dengan dua *pointer*.
3. Mampu mengimplementasikan penambahan *node*.
4. Mampu melakukan penelusuran maju dan mundur.
5. Mampu menghapus *node* dalam berbagai kondisi.

1.3 Manfaat Praktikum

1. Memahami pengelolaan data dua arah dalam *linked list*.
2. Melatih penggunaan *pointer next* dan *prev*.

3. Mengetahui proses penambahan dan penghapusan *node* secara fleksibel.
4. Meningkatkan logika dalam pengolahan data berantai.
5. Menjadi dasar untuk struktur data lanjutan.

BAB II

PEMBAHASAN

2.1 Double Linked List

Double Linked List adalah struktur data yang terdiri dari kumpulan *node* yang saling terhubung dengan dua arah. Setiap *node* memiliki data, *pointer* ke *node* berikutnya (*next*), dan *pointer* ke *node* sebelumnya (*prev*). Berbeda dengan *Single Linked List* yang hanya memiliki satu arah, struktur ini memungkinkan traversal dilakukan dari depan ke belakang maupun sebaliknya.

Keunggulan *Double Linked List* terletak pada kemudahan dalam proses penelusuran dan manipulasi data, terutama saat menghapus atau menyisipkan *node* di tengah *list*. Operasi yang umum dilakukan meliputi penambahan *node* di awal, akhir, dan posisi tertentu, penelusuran maju dan mundur, serta penghapusan *node*.

Dalam praktikum ini, berbagai operasi tersebut diimplementasikan untuk menunjukkan bagaimana data dapat dikelola secara fleksibel. Dengan adanya dua *pointer*, proses manipulasi data menjadi lebih efisien karena tidak perlu menelusuri ulang dari awal seperti pada *Single Linked List*.

2.2 Kode Pemograman

2.2.1 Class NodeDLL

```
2 package pekan6_2511531010;
3
4 public class NodeDLL_2511531010 {
5     int data_1010; // data
6     NodeDLL_2511531010 next_1010; // Pointer ke next node
7     NodeDLL_2511531010 prev_1010; // Pointer ke previous node
8
9     // Konstruktor
10    public NodeDLL_2511531010(int data_1010) {
11        this.data_1010 = data_1010;
12        this.next_1010 = null;
13        this.prev_1010 = null;
14    }
15 }
```

Gambar 2.1

Class ini merupakan dasar dari *Double Linked List* yang berfungsi sebagai *node*. Di dalamnya terdapat variabel data untuk menyimpan nilai, serta dua *pointer* yaitu *next* untuk menunjuk ke *node* berikutnya dan *prev* untuk menunjuk ke *node* sebelumnya. Konstruktor digunakan untuk menginisialisasi data dan mengatur kedua *pointer* bernilai *null* saat *node* pertama kali dibuat.

2.2.2 Class InsertDLL

```
1 package pekan6_2511531010;
2
3 public class InsertDLL_2511531010 {
4     // menambahkan node di awal DLL
5     static NodeDLL_2511531010 insertBegin(NodeDLL_2511531010
6     head_1010, int data_1010) {
7         // buat node baru
8         NodeDLL_2511531010 new_node_1010 = new
9         NodeDLL_2511531010(data_1010);
10        // jadikan pointer nextnya head
11        new_node_1010.next_1010 = head_1010;
12        // jadikan pointer prev head ke new_node
13        if (head_1010 != null) {
14            head_1010.prev_1010 = new_node_1010;
15        }
16        return new_node_1010;
17    }
18    // fungsi menambahkan node di akhir
19    public static NodeDLL_2511531010 insertEnd(NodeDLL_2511531010
20    head_1010, int newData_1010) {
21        // buat node baru
22        NodeDLL_2511531010 newNode_1010 = new
23        NodeDLL_2511531010(newData_1010);
24        // jika dll null jadikan head
25        if (head_1010 == null) {
26            head_1010 = newNode_1010;
27        }
28        else {
29            NodeDLL_2511531010 curr_1010 = head_1010;
30            while (curr_1010.next_1010 != null) {
31                curr_1010 = curr_1010.next_1010;
32            }
33            curr_1010.next_1010 = newNode_1010;
34            newNode_1010.prev_1010 = curr_1010;
35        }
36        return head_1010;
37    }
38    // fungsi menambahkan node di posisi tertentu
```

```

35     public static NodeDLL_2511531010
insertAtPosition(NodeDLL_2511531010 head_1010, int pos_1010, int
new_data_1010) {
36         // buat node baru
37         NodeDLL_2511531010 new_node_1010 = new
NodeDLL_2511531010(new_data_1010);
38         if (pos_1010 == 1) {
39             new_node_1010.next_1010 = head_1010;
40             if (head_1010 != null) {
41                 head_1010.prev_1010 = new_node_1010;
42             }
43             head_1010 = new_node_1010;
44             return head_1010;
45         }
46         NodeDLL_2511531010 curr_1010 = head_1010;
47         for (int i_1010 = 1; i_1010 < pos_1010 - 1 && curr_1010 !=
null; ++i_1010) {
48             curr_1010 = curr_1010.next_1010;
49         }
50         if (curr_1010 == null) {
51             System.out.println("Posisi tidak ada");
52             return head_1010;
53         }
54         new_node_1010.prev_1010 = curr_1010;
55         new_node_1010.next_1010 = curr_1010.next_1010;
56         curr_1010.next_1010 = new_node_1010;
57
58         if (new_node_1010.next_1010 != null) {
59             new_node_1010.next_1010.prev_1010 = new_node_1010;
60         }
61
62         return head_1010;
63     }
64     public static void printList(NodeDLL_2511531010 head_1010) {
65         NodeDLL_2511531010 curr_1010 = head_1010;
66         while (curr_1010 != null) {
67             System.out.print(curr_1010.data_1010 + " <-> ");
68             curr_1010 = curr_1010.next_1010;
69         }
70         System.out.println();
71     }
72     public static void main(String[] args) {
73         // membuat dll 2 <-> 3 <-> 5
74         NodeDLL_2511531010 head_1010 = new NodeDLL_2511531010(2);
75         head_1010.next_1010 = new NodeDLL_2511531010(3);
76         head_1010.next_1010.prev_1010 = head_1010;
77         head_1010.next_1010.next_1010 = new NodeDLL_2511531010(5);
78         head_1010.next_1010.next_1010.prev_1010 =
head_1010.next_1010;
79         // cetak DLL awal
80         System.out.print("DLL Awal: ");
81         printList(head_1010);
82         // tambah 1 di awal
83         head_1010 = insertBegin(head_1010, 1);

```



```

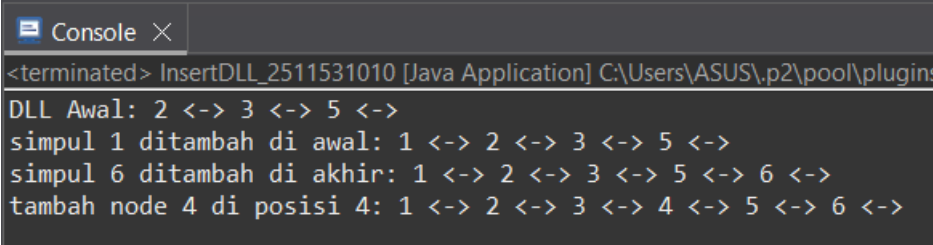
84     System.out.print("simpul 1 ditambah di awal: ");
85     printList(head_1010);
86     // tambah 6 di akhir
87     System.out.print("simpul 6 ditambah di akhir: ");
88     int data_1010 = 6;
89     head_1010 = insertEnd(head_1010, data_1010);
90     printList(head_1010);
91
92     // tambah node 4 di posisi 4
93     System.out.print("tambah node 4 di posisi 4: ");
94     int data2_1010 = 4;
95     int pos_1010 = 4;
96     head_1010 = insertAtPosition(head_1010, pos_1010,
data2_1010);
97     printList(head_1010);
98 }
99 }

```

Gambar 2.2

Class ini digunakan untuk menambahkan *node* ke dalam *Double Linked List*. Terdapat *method* untuk menambah *node* di awal (*insertBegin*), di akhir (*insertEnd*), dan pada posisi tertentu (*insertAtPosition*). Selain itu, terdapat *method* *printList* untuk menampilkan isi *list*. Pada bagian *main*, program membuat *list* awal kemudian melakukan beberapa operasi penambahan *node*.

2.2.3 Output InsertDLL



```

<terminated> InsertDLL_2511531010 [Java Application] C:\Users\ASUS\p2\pool\plugins
DLL Awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

```

Gambar 2.3

Output menampilkan kondisi awal DLL, kemudian menampilkan perubahan setelah penambahan *node* di awal, akhir, dan posisi tertentu. Setiap langkah menunjukkan bagaimana urutan data bertambah dan berubah sesuai operasi yang dilakukan.

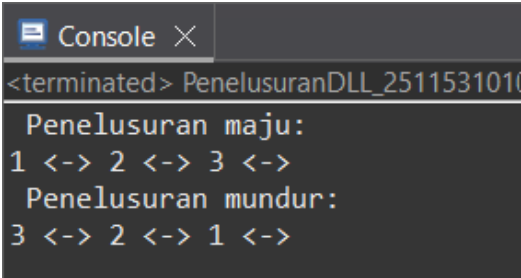
2.2.4 Class PenelusuranDLL

```
2 package pekan6_2511531010;
3
4 public class PenelusuranDLL_2511531010 {
5     //fungsi penelusuran maju
6     static void forwardTraversal_1010(NodeDLL_2511531010 head_1010)
7     {
8         // memulai penelusuran dari head
9         NodeDLL_2511531010 curr_1010 = head_1010;
10        //lanjutkan sampai akhir
11        while (curr_1010 != null) {
12            //print data
13            System.out.print(curr_1010.data_1010 + " <-> ");
14            //pindah ke node berikutnya
15            curr_1010 = curr_1010.next_1010;
16        }
17        //print spasi
18        System.out.println();
19    }
20    // fungsi penelusuran mundur
21    static void backwardTraversal_1010(NodeDLL_2511531010
22    tail_1010) {
23        // mulai dari akhir
24        NodeDLL_2511531010 curr_1010 = tail_1010;
25        //lanjut sampai head
26        while (curr_1010 != null) {
27            //cetak data
28            System.out.print(curr_1010.data_1010 + " <-> ");
29            // pindah ke node sebelumnya
30            curr_1010 = curr_1010.prev_1010;
31        }
32        // cetak spasi
33        System.out.println();
34    }
35    public static void main(String[] args) {
36        // cetak DLL
37        NodeDLL_2511531010 head_1010 = new NodeDLL_2511531010(1);
38        NodeDLL_2511531010 second_1010 = new NodeDLL_2511531010(2);
39        NodeDLL_2511531010 third_1010 = new NodeDLL_2511531010(3);
40
41        head_1010.next_1010 = second_1010;
42        second_1010.prev_1010 = head_1010;
43        second_1010.next_1010 = third_1010;
44        third_1010.prev_1010 = second_1010;
45
46        System.out.println(" Penelusuran maju:");
47        forwardTraversal_1010 (head_1010);
48
49        System.out.println(" Penelusuran mundur:");
50        backwardTraversal_1010 (third_1010);
51    }
52 }
```

Gambar 2.4

Class ini digunakan untuk melakukan penelusuran data dalam *Double Linked List*. Terdapat dua *method*, yaitu *forwardTraversal* untuk menelusuri dari depan ke belakang dan *backwardTraversal* untuk menelusuri dari belakang ke depan menggunakan *pointer prev*.

2.2.5 Output PenelusuranDLL



```
<terminated> PenelusuranDLL_2511531010
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->
```

Gambar 2.5

Output menampilkan hasil penelusuran maju dan mundur. Pada penelusuran maju, data ditampilkan dari *node* pertama hingga terakhir, sedangkan pada penelusuran mundur, data ditampilkan dari *node* terakhir ke *node* pertama.

2.2.6 Class HapusDLL

```
2 package pekan6_2511531010;
3
4 public class HapusDLL_2511531010 {
5     // fungsi menghapus node awal
6     public static NodeDLL_2511531010
7     delHead_1010(NodeDLL_2511531010 head_1010) {
8         if (head_1010 == null) {
9             return null;
10        }
11        head_1010 = head_1010.next_1010;
12        if (head_1010 != null) {
13            head_1010.prev_1010 = null;
14        }
15        return head_1010;
16    }
17    // fungsi menghapus di akhir
18    public static NodeDLL_2511531010
19    dellast_1010(NodeDLL_2511531010 head_1010) {
20        if (head_1010 == null) {
21            return null;
22        }
23    }
```

```

21         if (head_1010.next_1010 == null) {
22             return null;
23         }
24         NodeDLL_2511531010 curr_1010 = head_1010;
25         while (curr_1010.next_1010 != null) {
26             curr_1010 = curr_1010.next_1010;
27         }
28         // update pointer previous node
29         if (curr_1010.prev_1010 != null) {
30             curr_1010.prev_1010.next_1010 = null;
31         }
32         return head_1010;
33     }
34     // fungsi menghapus node posisi tertentu
35     public static NodeDLL_2511531010 delPos_1010(NodeDLL_2511531010
head_1010, int pos_1010) {
36         // jika DLL kosong
37         if (head_1010 == null) {
38             return head_1010;
39         }
40         NodeDLL_2511531010 curr_1010 = head_1010;
41         // telusuri sampai ke node yang akan dihapus
42         for (int i_1010 = 1; curr_1010 != null && i_1010 <
pos_1010; i_1010++) {
43             curr_1010 = curr_1010.next_1010;
44         }
45         // jika posisi tidak ditemukan
46         if (curr_1010 == null) {
47             return head_1010;
48         }
49         // update pointer
50         if (curr_1010.prev_1010 != null) {
51             curr_1010.prev_1010.next_1010 = curr_1010.next_1010;
52         }
53         if (curr_1010.next_1010 != null) {
54             curr_1010.next_1010.prev_1010 = curr_1010.prev_1010;
55         }
56         // jika yang dihapus head
57         if (head_1010 == curr_1010) {
58             head_1010 = curr_1010.next_1010;
59         }
60         return head_1010;
61     }
62     // fungsi mencetak DLL
63     public static void printList_1010 (NodeDLL_2511531010
head_1010) {
64         NodeDLL_2511531010 curr_1010 = head_1010;
65         while (curr_1010 != null) {
66             System.out.print(curr_1010.data_1010 + " <-> ");
67             curr_1010 = curr_1010.next_1010;
68         }
69         System.out.println();
70     }
71     public static void main(String[] args) {

```

```

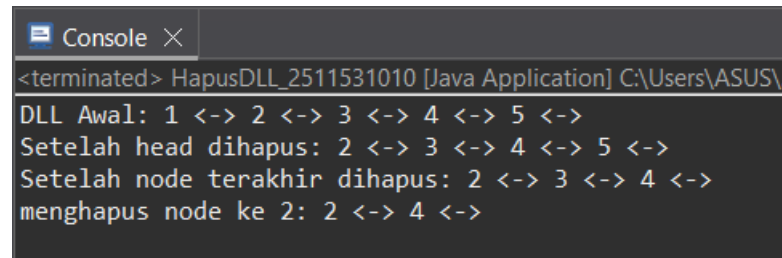
72      // buat sebuah DLL
73      NodeDLL_2511531010 head_1010 = new NodeDLL_2511531010(1);
74      head_1010.next_1010 = new NodeDLL_2511531010(2);
75      head_1010.next_1010.prev_1010 = head_1010;
76      head_1010.next_1010.next_1010 = new NodeDLL_2511531010(3);
77      head_1010.next_1010.next_1010.prev_1010 =
head_1010.next_1010;
78      head_1010.next_1010.next_1010.next_1010 = new
NodeDLL_2511531010(4);
79      head_1010.next_1010.next_1010.next_1010.prev_1010 =
head_1010.next_1010.next_1010;
80      head_1010.next_1010.next_1010.next_1010.next_1010 = new
NodeDLL_2511531010(5);
81      head_1010.next_1010.next_1010.next_1010.next_1010.prev_1010
= head_1010.next_1010.next_1010.next_1010;
82
83      System.out.print("DLL Awal: ");
84      printList_1010(head_1010);
85
86      System.out.print("Setelah head dihapus: ");
87      head_1010 = delHead_1010(head_1010);
88      printList_1010(head_1010);
89
90      System.out.print("Setelah node terakhir dihapus: ");
91      head_1010 = delLast_1010(head_1010);
92      printList_1010(head_1010);
93
94      System.out.print("menghapus node ke 2: ");
95      head_1010 = delPos_1010(head_1010, 2);
96
97      printList_1010(head_1010);
98  }
99 }

```

Gambar 2.6

Class ini digunakan untuk menghapus *node* dalam *Double Linked List*. Terdapat *method* untuk menghapus *node* di awal (*delHead*), di akhir (*delLast*), dan pada posisi tertentu (*delPos*). Selain itu, terdapat *method* untuk mencetak isi *list* sebelum dan sesudah penghapusan.

2.2.7 Output HapusDLL



```
Console X
<terminated> HapusDLL_2511531010 [Java Application] C:\Users\ASUS\
DLL Awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->
Setelah head dihapus: 2 <-> 3 <-> 4 <-> 5 <->
Setelah node terakhir dihapus: 2 <-> 3 <-> 4 <->
menghapus node ke 2: 2 <-> 4 <->
```

Gambar 2.7

Output menampilkan kondisi awal DLL, kemudian menunjukkan perubahan setelah *node* dihapus di bagian awal, akhir, dan posisi tertentu. Setiap langkah memperlihatkan bagaimana data berkurang dan struktur *list* tetap terhubung dengan benar.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa *Double Linked List* merupakan struktur data yang lebih fleksibel dibandingkan *Single Linked List* karena memiliki dua *pointer* yang memungkinkan penelusuran dua arah. Hal ini mempermudah proses manipulasi data, terutama dalam penambahan dan penghapusan *node*.

Melalui berbagai operasi yang telah diimplementasikan, seperti penambahan, penelusuran, dan penghapusan *node*, dapat dipahami bahwa struktur ini sangat efektif digunakan untuk pengelolaan data yang dinamis. Selain itu, pemahaman tentang *pointer next* dan *prev* menjadi hal penting dalam menjaga keterhubungan antar *node*.

Dengan demikian, praktikum ini membantu meningkatkan pemahaman tentang *Double Linked List* serta melatih kemampuan dalam mengimplementasikannya dalam program Java, yang dapat menjadi dasar untuk pengembangan program yang lebih kompleks.

DAFTAR PUSTAKA

- [1] Oracle Corporation. 2023. *Java Documentation*. Diakses dari <https://docs.oracle.com/javase/8/docs/>

TUGAS PRAKTIKUM

Soal:

1. Deskripsi Tugas

Buatlah program sederhana untuk mengelola playlist lagu menggunakan Double Linked List. Setiap lagu adalah node yang memiliki pointer ke lagu sebelumnya (prev) dan lagu berikutnya (next). Program harus memungkinkan pengguna menambah lagu, menghapus lagu, dan melihat daftar lagu dari dua arah.

2. Aturan Penamaan (Wajib)

Ikuti aturan yang sama seperti praktikum sebelumnya:

- Nama Kelas: pakai NIM lengkap. Contoh: Lagu_2411531005 dan Musik_2411531005
- Nama variabel & method: pakai 4 digit terakhir NIM. Contoh: judul_1005, tambahLagu_1005()
- Pointer: next_1005 dan prev_1005 juga wajib pakai akhiran NIM

3. Struktur yang Dibuat

Kelas Lagu_NIM

Atribut yang harus ada:

- judul (String)
- penyanyi (String)
- next (pointer ke Lagu)
- prev (pointer ke Lagu)

Method:

- constructor
- getter
- setter

Kelas Musik_NIM

- Buat 5 operasi utama saja (nama method diakhiri 4 digit NIM):

1. tambahLagu_xxxx() : menambah lagu baru di AKHIR playlist
2. hapusLaguAwal_xxxx() : menghapus lagu pertama (head)
3. tampilMaju_xxxx() : menampilkan semua lagu dari awal ke akhir
4. tampilMundur_xxxx() : menampilkan semua lagu dari akhir ke awal (wajib DLL)
5. cariLagu_xxxx(judul) : mencari lagu berdasarkan judul (tidak case-sensitive)

Kode Pemograman:

1. Class Lagu_2511531010

```
2. package pekan6_2511531010;
3.
4. public class Lagu_2511531010 {
5.     String judul_1010;
6.     String penyanyi_1010;
7.     Lagu_2511531010 next_1010;
8.     Lagu_2511531010 prev_1010;
9.
10.    // constructor
11.    public Lagu_2511531010(String judul_1010, String
    penyanyi_1010) {
12.        this.judul_1010 = judul_1010;
13.        this.penyanyi_1010 = penyanyi_1010;
14.        this.next_1010 = null;
15.        this.prev_1010 = null;
16.    }
17.    // getter
18.    public String getJudul_1010() {
19.        return judul_1010;
20.    }
21.    public String getPenyanyi_1010() {
22.        return penyanyi_1010;
23.    }
24.    // setter
25.    public void setJudul_1010(String judul) {
26.        this.judul_1010 = judul;
27.    }
28.    public void setPenyanyi_1010(String penyanyi) {
29.        this.penyanyi_1010 = penyanyi;
30.    }
31. }
```

2. Class Musik_2511531010

```
3. package pekan6_2511531010;
4.
5. import java.util.Scanner;
6.
7. public class Musik_2511531010 {
8.
9.     Lagu_2511531010 head_1010, tail_1010;
10.
11.    // 1. tambah lagu di akhir
12.    void tambahLagu_1010(String judul_1010, String penyanyi_1010)
    {
13.        Lagu_2511531010 baru_1010 = new
    Lagu_2511531010(judul_1010, penyanyi_1010);
14.
15.        if (head_1010 == null) {
```

```

16.         head_1010 = tail_1010 = baru_1010;
17.     } else {
18.         tail_1010.next_1010 = baru_1010;
19.         baru_1010.prev_1010 = tail_1010;
20.         tail_1010 = baru_1010;
21.     }
22.     System.out.println("Lagu berhasil ditambahkan!");
23. }
24. // 2. hapus lagu awal
25. void hapusLaguAwal_1010() {
26.     if (head_1010 == null) {
27.         System.out.println("Playlist kosong!");
28.     } else {
29.         System.out.println("Menghapus: " +
30. head_1010.judul_1010);
31.         head_1010 = head_1010.next_1010;
32.         if (head_1010 != null) {
33.             head_1010.prev_1010 = null;
34.         } else {
35.             tail_1010 = null;
36.         }
37.     }
38. }
39. // 3. tampil maju
40. void tampilMaju_1010() {
41.     if (head_1010 == null) {
42.         System.out.println("Playlist kosong!");
43.     } else {
44.         Lagu_2511531010 temp_1010 = head_1010;
45.         while (temp_1010 != null) {
46.             System.out.println(temp_1010.judul_1010 + " - " +
47. temp_1010.penyanyi_1010);
48.             temp_1010 = temp_1010.next_1010;
49.         }
50.     }
51. // 4. tampil mundur
52. void tampilMundur_1010() {
53.     if (tail_1010 == null) {
54.         System.out.println("Playlist kosong!");
55.     } else {
56.         Lagu_2511531010 temp_1010 = tail_1010;
57.         while (temp_1010 != null) {
58.             System.out.println(temp_1010.judul_1010 + " - " +
59. temp_1010.penyanyi_1010);
60.             temp_1010 = temp_1010.prev_1010;
61.         }
62.     }
63. // 5. cari lagu
64. void cariLagu_1010(String judul_1010) {
65.     if (head_1010 == null) {
66.         System.out.println("Playlist kosong!");

```

```

67.         return;
68.     }
69.     Lagu_2511531010 temp_1010 = head_1010;
70.     boolean ditemukan_1010 = false;
71.
72.     while (temp_1010 != null) {
73.         if
74.         (temp_1010.judul_1010.equalsIgnoreCase(judul_1010)) {
75.             System.out.println("Lagu ditemukan:");
76.             System.out.println(temp_1010.judul_1010 + " - " +
77. temp_1010.penyanyi_1010);
78.             ditemukan_1010 = true;
79.             break;
80.         }
81.         temp_1010 = temp_1010.next_1010;
82.     }
83.     if (!ditemukan_1010) {
84.         System.out.println("Lagu tidak ditemukan.");
85.     }
86. }
87.
88. // MAIN MENU
89. public static void main(String[] args) {
90.     Scanner sc = new Scanner(System.in);
91.     Musik_2511531010 musik_1010 = new Musik_2511531010();
92.
93.     int pilihan_1010;
94.
95.     do {
96.         System.out.println("=== Playlist Musik NIM:
97. 2511531010 ===");
98.         System.out.println("1. Tambah Lagu");
99.         System.out.println("2. Hapus Lagu Pertama");
100.        System.out.println("3. Lihat Playlist (Maju)");
101.        System.out.println("4. Lihat Playlist (Mundur)");
102.        System.out.println("5. Cari Lagu");
103.        System.out.println("6. Keluar");
104.        System.out.print("Pilihan: ");
105.        pilihan_1010 = sc.nextInt();
106.        sc.nextLine();
107.
108.        switch (pilihan_1010) {
109.            case 1:
110.                System.out.print("Judul: ");
111.                String judul_1010 = sc.nextLine();
112.                System.out.print("Penyanyi: ");
113.                String penyanyi_1010 = sc.nextLine();
114.                musik_1010.tambahLagu_1010(judul_1010,
115.                penyanyi_1010);
116.                break;
117.
118.            case 2:
119.                musik_1010.hapusLaguAwal_1010();
120.                break;

```

```

117.
118.         case 3:
119.             musik_1010.tampilMaju_1010();
120.             break;
121.
122.         case 4:
123.             musik_1010.tampilMundur_1010();
124.             break;
125.
126.         case 5:
127.             System.out.print("Masukkan judul lagu:
128. ");
129.             String cari = sc.nextLine();
130.             musik_1010.carilagu_1010(cari);
131.             break;
132.
133.         case 6:
134.             System.out.println("Program selesai.");
135.             break;
136.
137.         default:
138.             System.out.println("Pilihan tidak
139. valid!");
140.     }
141. } while (pilihan_1010 != 6);
142. sc.close();

```

3. Output



```

<terminated> Musik_2511531010 [Java Application] C
=== Playlist Musik NIM: 2511531010 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: like JENNIE
Penyanyi: JENNIE
Lagu berhasil ditambahkan!
=== Playlist Musik NIM: 2511531010 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Toxic till the end
Penyanyi: Rosé
Lagu berhasil ditambahkan!

```

```

=== Playlist Musik NIM: 2511531010 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
like JENNIE - JENNIE
Toxic till the end - Rosé
=== Playlist Musik NIM: 2511531010 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
Toxic till the end - Rosé
like JENNIE - JENNIE
=== Playlist Musik NIM: 2511531010 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: like JENNIE
Lagu ditemukan:
like JENNIE - JENNIE
=== Playlist Musik NIM: 2511531010 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Menghapus: like JENNIE
=== Playlist Musik NIM: 2511531010 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 6
Program selesai.

```

Tugas ini bertujuan untuk mengimplementasikan *Double Linked List* (DLL) dalam bentuk simulasi *playlist* lagu sederhana. Pada struktur ini, setiap data lagu disimpan dalam sebuah *node* yang saling terhubung melalui dua *pointer*, yaitu *next* (ke *node* berikutnya) dan *prev* (ke *node* sebelumnya), sehingga membentuk rantai data yang dapat diakses dari dua arah.

Setiap *node* (*class* Lagu_2511531010) memiliki atribut berupa judul lagu, nama penyanyi, serta *pointer next* dan *prev*. Dengan adanya dua *pointer*

ini, data dalam *playlist* dapat ditelusuri baik dari awal ke akhir maupun dari akhir ke awal.

Konsep utama yang digunakan adalah pengelolaan data secara berurutan dalam *playlist*, di mana lagu yang ditambahkan akan berada di bagian akhir. Hal ini diimplementasikan dengan:

- *Insert* di akhir (*tail*) saat menambahkan lagu baru ke *playlist*.
- *Traversal* dua arah, yaitu dari depan ke belakang menggunakan *pointer next*, dan dari belakang ke depan menggunakan *pointer prev*.

Program juga menyediakan beberapa operasi penting, yaitu:

- *tambahLagu* untuk menambahkan lagu ke dalam *playlist*.
- *hapusLaguAwal* untuk menghapus lagu pertama dalam *playlist*.
- *tampilMaju* untuk menampilkan seluruh lagu dari awal ke akhir.
- *tampilMundur* untuk menampilkan lagu dari akhir ke awal (ciri khas *Double Linked List*).
- *cariLagu* untuk mencari lagu berdasarkan judul tanpa membedakan huruf besar dan kecil.